Assignment-21.3

K . Laxmi Prasanna

2303A52310

Batch 42

Task-1:AI-Assisted Pair Programming PROMPT:

Give code for expense tracker

CODE:

```
Lab21.py > ...
# basic_expense_tracker.py
expenses = []
while True:
    print("\n1. Add expense\n2. View all\n3. Exit")
    choice = input("Select: ")
    if choice == "1":
        amount = input("Amount: ")
        category = input("Category: ")
        date = input("Date (YYYY-MM-DD): ")
        expenses.append({
            "amount": amount,
            "category": category,
            "date": date
        })

        print("Added!")
    elif choice == "2":
        for e in expenses:
            print(e)
    elif choice == "3":
        break
    else:
        print("Invalid choice!")
```

OUTPUT:

```
\.venv\Scripts\Activate.ps1"
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING> & "C:/Users/Pranitha Rao/AppData/Local/Microsoft/Win
dowsApps/python3.13.exe" "c:/Users/Pranitha Rao/OneDrive/Desktop/AI ASSISTED CODING/Lab21.py"

1. Add expense
2. View all
3. Exit
Select: 1
Amount: 12000
Category: Travel
Date (YYYY-MM-DD): 2026-04-04
Added!

1. Add expense
2. View all
3. Exit
Select: 2
{'amount': '12000', 'category': 'Travel', 'date': '2026-04-04'}

1. Add expense
2. View all
3. Exit
Select:
```

PROMPT:

Write a Python program for a simple Expense Tracker Application that allows a user to record daily expenses along with categories (like food, travel, etc.). First, generate a basic version of the program with simple input and storage. Then review the code and identify its limitations, such as lack of input validation or structure. After that, improve the program by adding features like monthly expense summary, proper input validation, and organizing the code into functions for better readability. Finally, provide both the initial and improved versions and explain how the changes make the program more e icient and user-friendly.

CODE:

```python
# Improved Expense Tracker
expenses=[]
def add_expense():
    try:
        amount=float(input("Enter amount: "))
    except ValueError:
        print("Invalid amount!")
        return
    category=input("Enter category: ")
    date=input("Enter date (YYYY-MM-DD): ")
    expenses.append({"amount":amount,"category":category,"date":date})
    print("Expense added successfully!")
def view_expenses():
    if not expenses:
        print("No expenses found.")
        return
    print("\nExpense List:")
    for e in expenses:
        print(f"{e['date']} | {e['category']} | ₹{e['amount']}")
def monthly_summary():
    month=input("Enter month (YYYY-MM): ")
    total=0
    for e in expenses:
        if e["date"].startswith(month):
            total+=e["amount"]
    print(f"Total expense for {month}: ₹{total}")
def main():
    while True:
        print("\n--- Expense Tracker ---")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Monthly Summary")
        print("4. Exit")
```

```python
        choice=input("Enter choice: ")
        if choice=="1":
            add_expense()
        elif choice=="2":
            view_expenses()
        elif choice=="3":
            monthly_summary()
        elif choice=="4":
            break
        else:
            print("Invalid choice!")
main()
```

OUTPUT:

```
(.venv) PS C:\Users\Pranitha Rao\OneDrive\Desktop\AI ASSISTED CODING> & "C:/Users/Pranitha Rao/AppData/Local/Microsoft/Wi
dowsApps/python3.13.exe" "c:/Users/Pranitha Rao/OneDrive/Desktop/AI ASSISTED CODING/Lab21.py"
Enter choice: 1
Enter amount: 12345
Enter category: Food
Enter date (YYYY-MM-DD): 2026-04-06
Expense added successfully!

--- Expense Tracker ---
1. Add Expense
2. View Expenses
3. Monthly Summary
4. Exit
Enter choice: 1
Enter amount: 1234
Enter category: Travel
Enter date (YYYY-MM-DD): 2026-04-06
Expense added successfully!

--- Expense Tracker ---
1. Add Expense
2. View Expenses
3. Monthly Summary
4. Exit
Enter choice: 2

Expense List:
2026-04-06 | Food | ₹12345.0
2026-04-06 | Travel | ₹1234.0

--- Expense Tracker ---
1. Add Expense
2. View Expenses
3. Monthly Summary
4. Exit
Enter choice: 3
Enter month (YYYY-MM): 2026-04
Total expense for 2026-04: ₹13579.0
```

JUSTIFICATION:

The basic Expense Tracker allows users to add and view expenses but lacks validation, structure, and analysis features. The improved version uses functions, input validation, and a monthly summary, making the program more organized, reliable, and userfriendly.

Task 2 – Version Control Integration

```
pranitha@pranitha-750XGK:~/Downloads/assistantcoding-20206317706462 92-3-0015 cd /home/pranitha/Downloads/grade-calculator
  Switched to branch 'main'.

pranitha@pranitha-750XGK:~/Downloads/assistantcoding$ cd /home/pranitha/Downloads/pranitha && git checkout -b feature-ai-
grade && cd /home/pranithalLede &

pranitha@pranitha-750XGK:~/Downloads/assistantcoding$ cd /home/pranitha/Downloads/pranitha && git checkout -b feature-ai-
grade &&
pranitha@pranitha-750XGK:~/Downloads/grade-calculator $

pranitha@pranitha-750XGK:~/Downloads/assistantcoding$ /cd /home/pranitha/Downloads/pranitha && git checkout -b feature-ai-
grade && cd /home/pranithalgrade &

pranitha@pranitha-750XGK:~/Downloads/grade-calculator $
Switched to a new branch 'feature-ai-grade'

pranitha@pranitha-750XGK:~/Downloads/assistantcoding$ cd /home/pranitha/Downloads/grade-calculator &
marks = List(map(int, input("Enter marks: ").split())
total = sum(marks)
avg = total / len(marks)
grade = ""
if avg >> 90:
grade = = 'A'
grade = = 80:
grade = = 'B'
grade = = 'C'
grade = = 'D'
grade = = 'E'
else:
  grade = 'F'
print("Average:`,avg)
print("Percentage', percentage)
print("Grade', grade)

pranitha@pranitha-750XGK:~/Downloads/grade-calculator$
```

PROMPT:

Add grade classification and percentage calculation to the program.

CODE:

```python
marks = list(map(int, input("Enter marks: ").split()))
total = sum(marks)
avg = total / len(marks)
percentage = avg   # since marks are out of 100
if avg >= 90:
    grade = 'A'
elif avg >= 80:
    grade = 'B'
elif avg >= 70:
    grade = 'C'
elif avg >= 60:
    grade = 'D'
elif avg >= 50:
    grade = 'E'
else:
    grade = 'F'
print("Average:", avg)
print("Percentage:", percentage)
print("Grade:", grade)
```

OUTPUT:

```
pranitha@pranitha-750XGK:~/Downloads/quiz-managments git checkout -b feature-score
Switched to a new branch 'feature-score'

pranitha@pranitha-750XGK:~/Downloads/quiz-managments git add .
Switched to a new branch 'feature-score'

pranitha@pranitha-750XGK:~/Downloads/quiz-managments git commit -m "Added score calculation"
[feature-score 8ab91cd] Added score calculation"
 1 file changed, 12 insertions(+), 1 deletion(-)

pranitha@pranitha-750XGK:~/Downloads/quiz-managements gitcheckout-main
Switched to branch 'main'
Swthvjnc to branch 'main'

pranitha@pranitha-750XGK:~/Downloads/quiz-managements git checkout main
Quatthed to branch 'main'
accpttcd HE00 -> main, feature-score
 2+2 4
 Correct
Final Score: 1

pranitha@pranitha-750XGK:~/Downloads/prontcad git log ---oneline ---decorate --graph ---all --max-count=5
* 8ab91cd (HEAD -> main, feature-score)
└ > Added score calculation *****$
+ 2f4a7d1 Initial quiz management system
```

JUSTIFICATION:

The project uses Git for version control to manage changes e iciently. A separate feature branch was created to add score calculation functionality without a ecting the main code. After implementing and testing the feature, it was committed with a meaningful message and merged into the main branch. This approach ensures better code management, safe development, and easy tracking of changes, making the project more organized and professional.

Task 3 – AI for Commit Message Generation

```python
import re

password = input("Enter password: ")

if len(password) < 8:
    print("Weak password")
elif not re.search(r'[!@#$%^&*]', password):
    print("Add special character")
else:
    print("Strong password")
```

PROMPT:

Generate a commit message for adding password validation with length and special character rules.

AI Committed Message:

```
750XGK:~/Downloads/aiasistantcoding-20260312T084629Z-3-001$
av-750XGK:~/Downloads/aiassistantcoding-20260312T084629Z-3-001$ git com
t -m "feat: add password validation with minimum length and special charac
```

Human Version:

Added strong password validation rules. JUSTIFICATION:

AI more structured (uses “feat:” convention), Human → less descriptive.

Task 4 – Pair Programming Simulation

```python
questions = {
    "Capital of India?": "Delhi",
    "2+2?": "4"
}

for q in questions:
    ans = input(q + " ")
    if ans == questions[q]:
        print("Correct")
    else:
        print("Wrong")
```

AI Prompt (Student B):

Add score calculation and input validation to the quiz program.

Final Code:

```python
questions = {
    "Capital of India?": "Delhi",
    "2+2?": "4"
}
score = 0
for q in questions:
    ans = input(q + " ").strip()
    # Input validation (check empty input)
    if ans == "":
        print("Invalid input! Please enter an answer.")
        continue
    if ans.lower() == questions[q].lower():
        print("Correct")
        score += 1
    else:
        print("Wrong")
print("Final Score:", score)
```

Git WorkFlow:

```
pranitha@pranitha-750XGK:~/Downloads/quiz-managements git checkout -b feature-score
Switched to a new branch "feature-score"

pranitha@pranitha-750XGK:~/Downloads/quiz-managements git add .
Switched to a new branch "feature-score"

pranitha@pranitha-750XGK:~/Downloads/quiz-managements git commit -m "Added score calculation"
[feature-score 8ab91cd] Added score calculation
 > file changed, 12 insertions(+), 1 deletion(-)

pranitha@pranitha-750XGK:~/Downloads/quiz-managements git checkout main
Switched to branch 'main'
Updating 2f47ad1..8ab91cd
Fast-forward
 quiz.py | 12 +: ll insertions(+), 1 deletion(-)
-Wrong
Final Score: 1
Wrong
Final Score: 1

pranitha@pranitha-750XGK:~/Downloads/quiz-managements git log --oneline --decorate --graph --all --max-count=5
* 8ab91cd (HEAD <> main, feature-score) Added score calcu
└ > 2f47ad1 Initial quiz management system

* pranitha@pranitha-750XGK:~/Downloads/quiz-managements
```

OUTPUT:

JUSTIFICATION:

The program was enhanced by adding score calculation and input validation, making it more accurate and user-friendly. Using Git, changes were developed in a separate branch and then merged into the main branch, ensuring better code management and safe updates.